Freight Rate Prediction

Machine Learning Engineer take-home assessment, Spotter

Marwan Nasser · github.com/MarwanNasser999/freight-rate-prediction

1. Summary

The task is to predict the posted rate (USD) of truck loads. The labelled data covers **January–October 2025**; the 12,000 loads to predict are from **November–December 2025**. This is a forecasting problem, so every model was validated by training on the past and testing on the following two months.

Model	MAE Sep–Oct	MAE Jul–Aug
Baseline: median $/mile by equipment × distance	$83.0 (3.8%)	$87.6 (4.0%)
Final: two-part model (linear price level + LightGBM)	**$33.0 (1.4%)**	**$44.3 (1.8%)**

- **60% lower error than the baseline** on both test periods, with errors of about 1.3–1.5% across every equipment type and distance range.
- **Key discovery:** besides the daily market level, prices drift upward (~0.6%/month) and **climb ~4% in the last four weeks of every quarter**. The model is designed so these patterns can be carried into November–December.
- **Validation is leakage-checked:** all cleaning statistics are learned from training months only, and seven automated checks (including a shuffled-label test) pass.

Scores are on test loads with valid prices. About 1.4% of prices in the data are corrupted; including them in the test raises MAE to ~$89–96 for this model ($139 for the baseline). No model can predict those values.

2. Data and key findings

- **Validation is the future.** train_test.csv spans Jan 1 – Oct 31; validation.csv spans Nov 1 – Dec 31. There is no overlap, so a random split would not reflect the real task.
- **Price is mostly distance.** Rate per mile (median ~$2.15) is the stable quantity to model. Short trips cost more per mile ($2.74 under 250 mi vs $1.87 over 2,500 mi), and Reefer (+13%) and Flatbed (+8%) cost more than Dry Van.
- **market_index is a daily market level.** All loads on the same day share almost the same value (spread within a day ≈ 0.025). It has a weekly cycle (high Thursday, low Sunday) and market regimes (high Apr–Jul, a sharp drop in late July).
- **8 cities appear only in validation**, affecting 1,447 loads (12%). The model therefore relies on distance and coordinates, not city names, so it can price unseen cities.
- **quote_signal looked weak by correlation (0.10)** but improved the model when tested: its effect is non-linear (only the highest quotes matter) and mostly for Reefer and Flatbed.

The most important finding is the structure of the daily price level (each day's average rate per mile relative to what is typical for its distance and equipment):

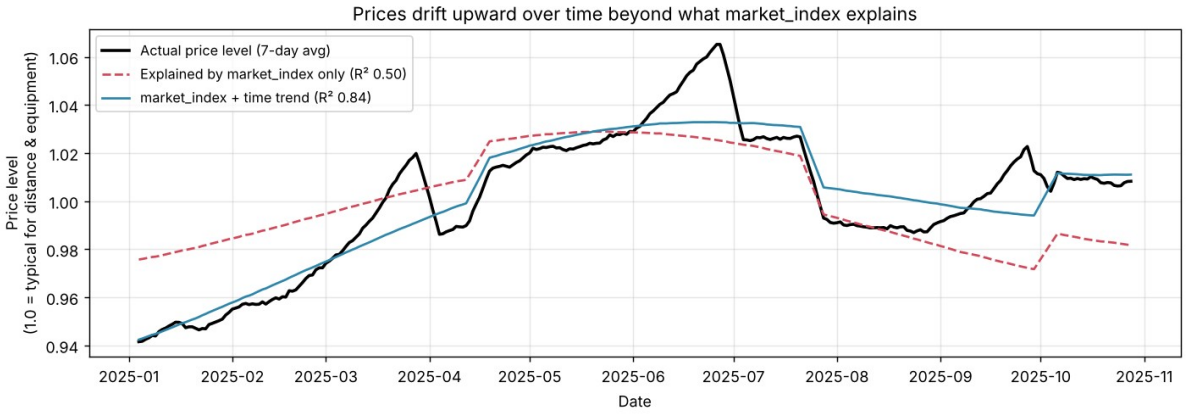


Figure 1. Daily price level: market_index alone explains 50% of its variation; adding a time trend explains 84%.

The remaining spikes appear at the end of **March, June and September**: prices ramp up over the last ~4 weeks of each quarter and reset on the 1st. Adding this **quarter-end ramp** raises the explained variation to **97%**. December is also a quarter end (and year end), so this matters directly for the forecast.

3. Data-quality issues and how they were addressed

Issue	Evidence	Fix
677 corrupted prices (1.4%)	Rates 0.15–0.5× or 2–5.5× the typical $/mile for the same equipment and distance, with an empty gap separating them from normal loads (0.8–1.3×)	Removed from **training only**. Every load is still predicted.
Negative weights (292 train, 145 validation)	Without the minus sign, they have the same distribution as normal weights; their prices are normal	Absolute value (sign error)
Missing weights (300 / 165)	Missing loads have normal prices, so missingness carries no signal	Median weight from training months
Missing market_index (374 / 249)	It is a daily value shared by all loads that day	That day's average market_index
Weight exactly 47,500 lb (1,191 loads)	Matches the legal cargo limit	Kept (a cap, not an error)
Coordinates shifted from real geography	Consistent per city; distance column agrees with prices	Kept as relative positions; distance never recomputed

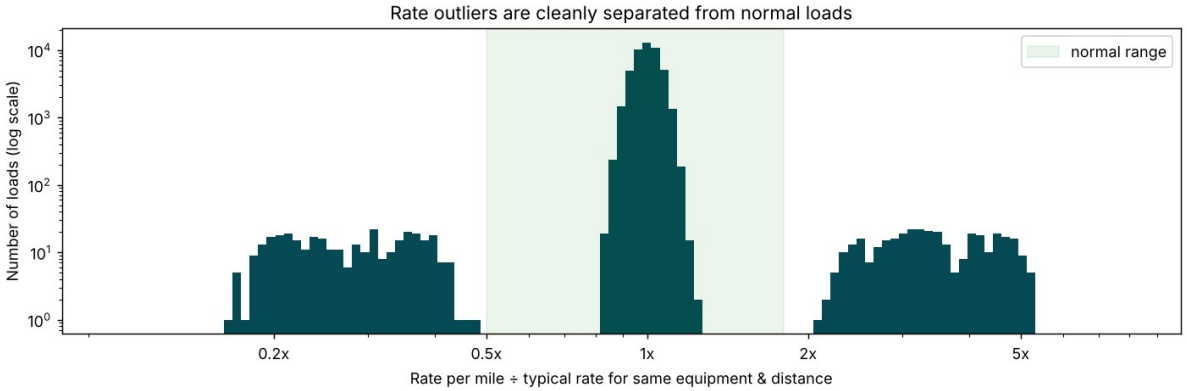


Figure 2. Each load's $/mile divided by the typical rate for its equipment and distance (log scales). Corrupted prices sit far from normal loads, so any cut-off in the gap selects the same 677 rows.

Removing the corrupted prices from training was tested, not assumed: keeping them roughly doubles the error (average MAE $73.0 vs $38.6).

4. Train/test split and validation approach

4.1 Why a time-based split

Spotter evaluates on November–December, a period the model has never seen. A random split would place loads from the same days (and the same market conditions) in both training and test, which makes the test too easy. To show this, the same LightGBM model was scored both ways:

Split	MAE	Interpretation
Random 80/20	$53	Looks good, but the model has already seen every day it is tested on
Time-based (train Jan–Aug, test Sep–Oct)	$105	Honest: the test period is entirely in the future

A random split would have overstated performance by about **2×**, so all decisions below use time-based splits.

4.2 The folds

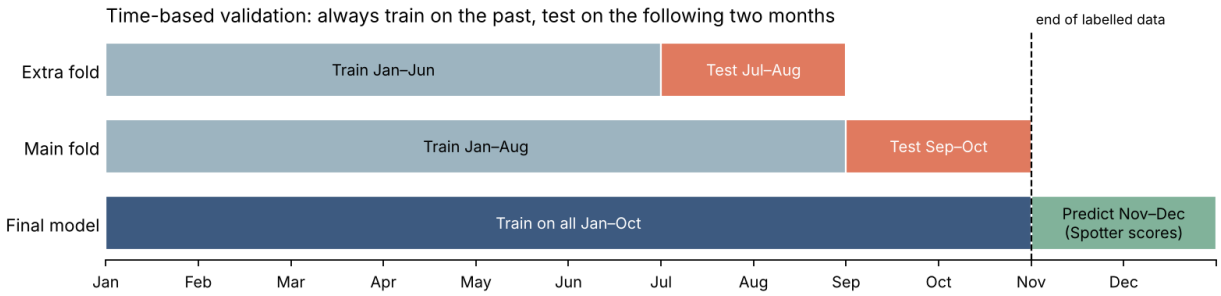


Figure 3. Two validation folds, each testing on the two months after the training period, then a final model trained on all labelled data.

- **Main fold: train Jan–Aug, test Sep–Oct.** Same two-month horizon as the real task, and Sep–Oct is in the same post-July market regime as Nov–Dec.
- **Extra fold: train Jan–Jun, test Jul–Aug.** A second, harder period (it contains the late-July market drop). A choice is only accepted if it helps on both folds, which protects against decisions that work by luck on one period.
- **Final model: retrain on all of Jan–Oct**, after every decision was frozen, so it learns from the most recent data before predicting Nov–Dec. The fold scores estimate how well this method performs.

4.3 Metrics

Spotter's metric is not stated, so three are reported: **MAE** (average error in dollars, the main metric), **MAPE** (average % error, fair across short and long trips) and **RMSE** (penalises large misses). **Bias** (total predicted ÷ total actual − 1) is also tracked to detect systematic over- or under-prediction. The main score uses test loads with valid prices; the score including corrupted prices is reported alongside.

4.4 Preventing leakage

The pipeline splits by date first, then learns every statistic (median weight, the outlier rule's typical rates) from the training months only. An automated audit (src/leakage_checks.py) runs seven checks, all passing:

Check	Result
1. Time order	Training always ends before testing starts; 0 shared load IDs between train/test and train/validation
2. Row-wise features	A load's features are identical whether computed alone or alongside all 48,000 loads
3. Train-only statistics	Found and fixed two small leaks (outlier rule and median weight used all months); impact $32.9 → $33.0
4. Shuffled-label test	Training on shuffled prices gives MAE $1,132 vs $33: no hidden path from inputs to the answer
5. Patterns found without test months	Trend and quarter-end ramp learned from Jan–Aug (+0.54%/month, +4.3%) match Jan–Oct (+0.59%, +4.1%)
6. No tuning optimism	Settings chosen on Jul–Aug only score $33.0 on Sep–Oct, the same as the final settings
7. Reproducible	Fixed random seed; repeated runs give identical results

market_index and quote_signal are provided for every validation load, so using them is legitimate: they are available at prediction time and are never prices.

5. Model and results

5.1 Why a two-part model

Prices have an upward trend, but tree models such as LightGBM cannot extrapolate beyond the time range they were trained on. A linear model can carry a trend forward but misses route-level detail. The final model combines both, predicting **log(rate per mile)**:

1. **Part 1, linear price level:** log distance, equipment, weight, market_index, months since Jan 1 (trend) and the quarter-end ramp (0 for most of a quarter, rising to 1 on its last day).
2. **Part 2, LightGBM correction** trained on Part 1's errors: distance, weight, equipment, pickup/delivery coordinates, travel direction and quote_signal. It has **no time features**, so it never needs to extrapolate.

Prediction = distance × exp(Part 1 + Part 2). Working in log space makes effects multiplicative ("Reefer is +13%") and guarantees positive prices. Part 1 is also readable: Reefer +13%, Flatbed +8%, +3% per 10,000 lb, +1.4% per +0.1 market_index, a **+0.6% per month** trend and up to **+4.1%** at a quarter end.

5.2 Comparison (MAE on valid test prices)

Approach	Sep–Oct	Jul–Aug	Average
Baseline: median $/mile by equipment × distance	$83.0	$87.6	$85.3
Linear model only (Part 1)	$57.6	$59.6	$58.6
LightGBM only, all features incl. time	$46.1	$76.3	$61.2
Two-part: linear + Random Forest	$35.6	$45.4	$40.5

Approach	Sep–Oct	Jul–Aug	Average
Two-part: linear + sklearn HistGradientBoosting	$34.2	$44.6	$39.4
Two-part: linear + LightGBM (final)	**$33.0**	**$44.3**	**$38.6**

LightGBM alone does well on Sep–Oct but poorly on Jul–Aug, which is the extrapolation problem made visible. The quarter-end ramp was the decisive feature: during development, without it the two-part model scored $98.9 on Jul–Aug, because the March and June ramps were mistaken for a steep trend.

5.3 Questions answered by tests

Question	Result	Decision
Remove corrupted prices from training?	Keeping them: $73.0; with Huber loss: $41.5	Remove
Weight recent months more?	Half-life 60 days: $40.5; 120 days: $39.5	No weighting
Absolute-error (L1) loss?	$39.0	Keep default
Add day of week?	Worse (market_index already carries the weekly cycle)	Not used
Add lane/city average prices?	~$0.5 gain, extra complexity and leakage risk	Not used
Tune LightGBM (24 settings, both folds)?	Best $38.6, worst $39.8: the model is not sensitive to settings	63 leaves, ≥50 loads per leaf, lr 0.03, 1,200 trees

5.4 Unseen cities

To simulate the 8 new cities in validation, 8 random cities were hidden from training (3 repeats) and the model was scored on Sep–Oct loads involving them: MAE **$38–42** on new cities vs ~$32 on seen cities, compared with **$85–95** for the baseline. Coordinates let the model price cities it has never seen.

6. Final predictions

The final model was retrained on all clean January–October data and used to predict the 12,000 validation loads. Their true prices are held by Spotter, so plausibility was checked instead (src/check_predictions.py, 6/6 passing): correct format; $/mile within the training range ($1.67–3.65); close to Sep–Oct prices per equipment (+0.3% to +1.3%, reflecting the trend and the Q4 ramp); 99.7% correlated with the simple baseline; loads with new cities behave like the rest; and all validation inputs are within training ranges.

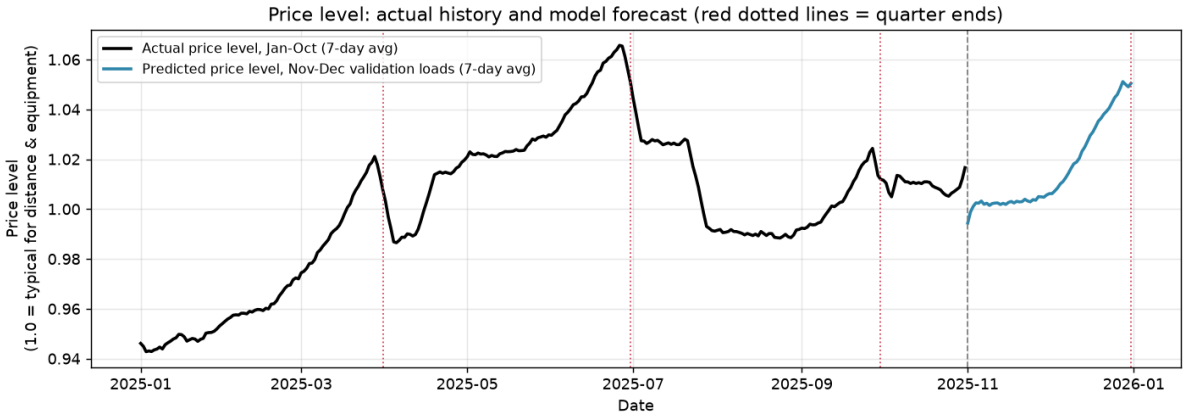


Figure 4. Actual price level (Jan–Oct) and the predicted price level of the Nov–Dec validation loads. The forecast is flat in November and climbs into the quarter end, like March, June and September.

7. December prediction chart

The chart fixes one load (Lexington → Fort Wayne, 360 miles, Dry Van, 32,000 lb) and changes only the date. The input file lacks three features the model uses, which were filled as follows:

- **market_index:** each December day's average from validation.csv (it is one value per day).
- **Coordinates:** the cities' coordinates as they appear in the data.
- **quote_signal:** the median quote for Dry Van loads. Any reasonable value moves predictions by less than 1%.

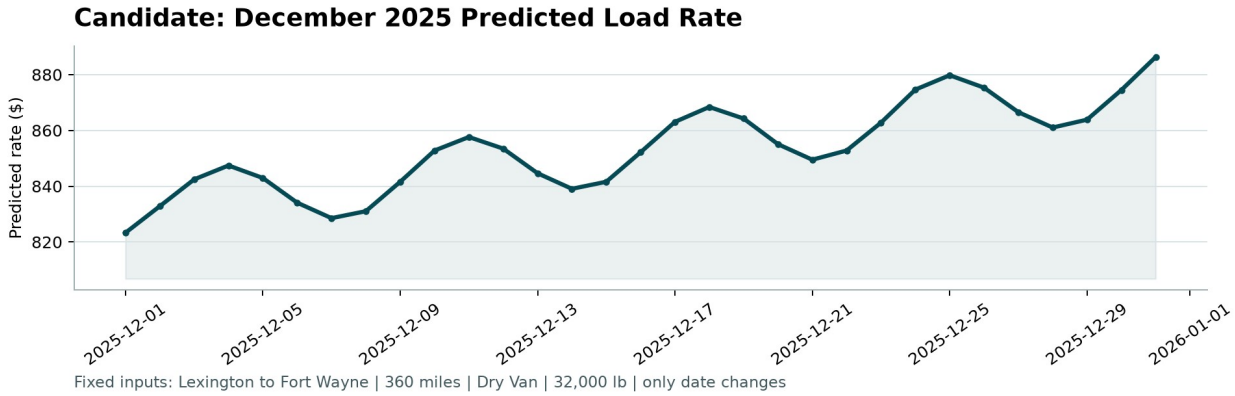


Figure 5. December 2025 predictions for the fixed load, produced by the provided score.py.

Predictions rise from **$823 on Monday Dec 1 to $886 on Wednesday Dec 31**. Because the load is fixed, only Part 1 changes with the date, so the shape has three explainable components:

1. **A weekly wave from market_index:** peaks every Thursday (Dec 4, 11, 18, 25) and dips around Sunday–Monday.
2. **A steady rise** from the upward trend (~0.6% per month).
3. **A stronger climb over the last four weeks** from the quarter-end (and year-end) ramp, so each weekly peak is higher than the last ($847 → $858 → $868 → $880) and the month ends at its highest point.

For reference, the most recent actual Dry Van loads on this lane (Sep–Oct) cost $778–865, so the December range is consistent with the history plus the year-end ramp.

8. Limitations and next steps

- **Trend extrapolation:** the upward trend is assumed to continue for two more months. A market reversal would make predictions slightly high. With more history, the trend could be modelled with a dampened or regularly re-estimated slope.
- **No December in training:** holiday effects (e.g. around Christmas) can only enter through market_index, which shows no holiday dip in the given data.
- **Corrupted labels in scoring:** if Nov–Dec contains ~1.4% corrupted prices like the training data, they will add an unavoidable error for any model.
- **Possible improvements:** lane-level history features with careful time-safe encoding, prediction intervals for pricing decisions, and monitoring the trend and quarter-end effects as new data arrives.

Appendix: reproducing the results

From the repository root, after installing requirements.txt:

```
python src/predict.py   (final model, validation_predictions.csv, December
predictions)
python score.py --predictions validation_predictions.csv --december-predictions
data/december_chart_inputs.csv
python src/experiments.py   (all comparisons in Section 5)
python src/leakage_checks.py   (Section 4.4)
python src/check_predictions.py   (Section 6)
```